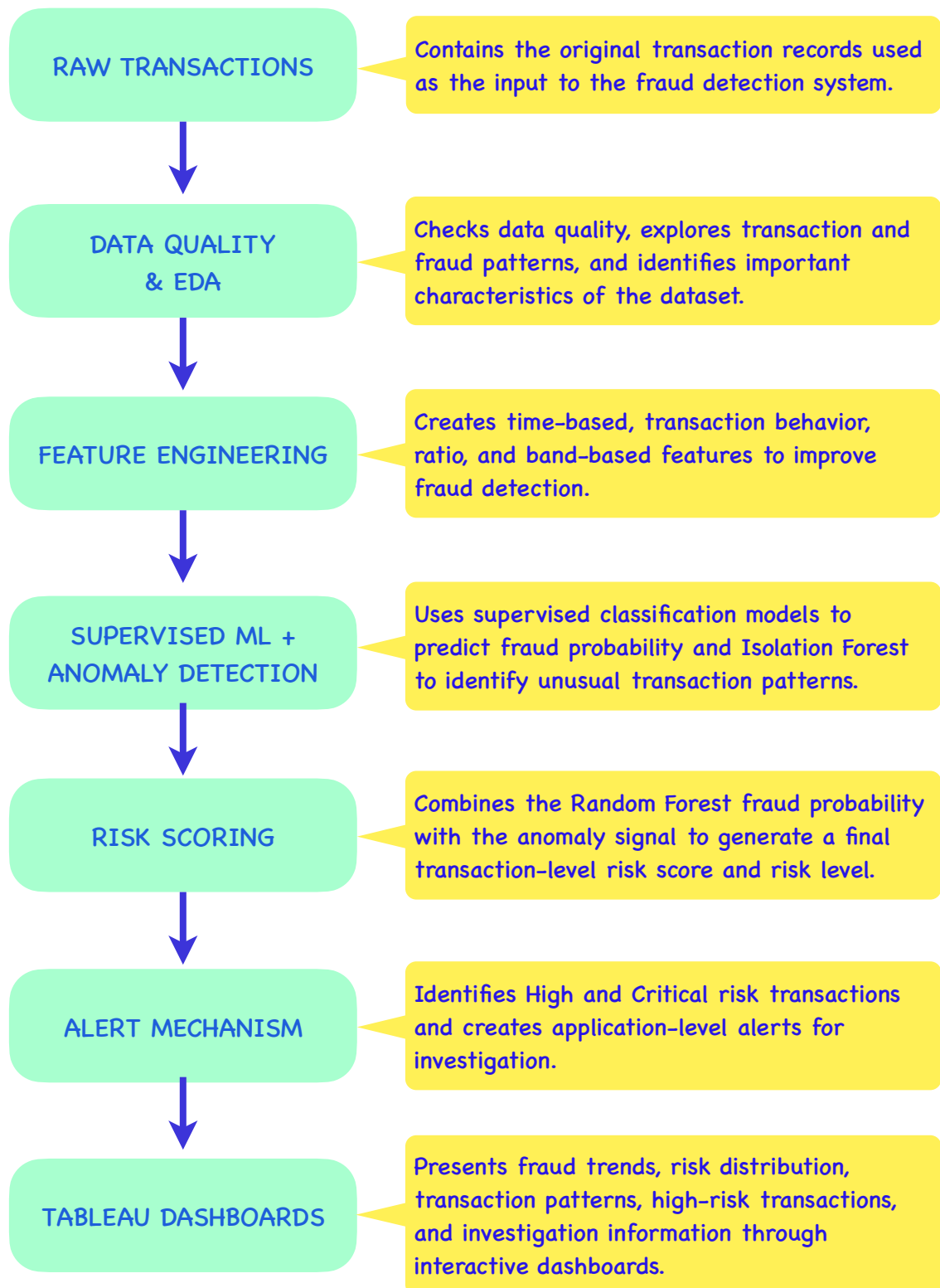


1. End-to-End Financial Fraud Detection & Risk Intelligence Architecture



2. Executive Summary

Financial Fraud Detection & Risk Intelligence System is an end-to-end machine learning project designed to identify potentially fraudulent financial transactions and provide transaction-level risk intelligence.

The system uses a combination of **supervised machine learning and anomaly detection** to analyze transaction behavior. Multiple classification models were evaluated, including Logistic Regression, Decision Tree, Random Forest, and XGBoost. Random Forest was selected as the final supervised model based on cross-validation performance, particularly its recall and F1-score for fraud detection.

An **Isolation Forest** model was also used to identify unusual transaction patterns independently of the supervised fraud prediction. The fraud probability and anomaly signal were then combined into a **Risk Score**, which categorizes transactions into Low, Medium, High, and Critical risk levels.

High- and Critical-risk transactions are flagged through an **application-level alert mechanism** for investigation. The resulting transaction and risk information is presented through interactive **Tableau dashboards**, providing views of fraud trends, transaction patterns, risk distribution, and high-risk transactions.

The completed system demonstrates an end-to-end workflow from **raw transaction data to fraud prediction, anomaly detection, risk scoring, alert generation, and business intelligence visualization**.

3. Business Problem

Financial transactions are exposed to different forms of fraudulent and suspicious activity. Detecting fraudulent transactions using only fixed rules can be difficult because transaction behavior can vary across customers, payment methods, locations, devices, transaction amounts, and time periods.

A fraud detection system therefore needs to identify both known fraud patterns and unusual transaction behavior, while also providing a practical way to prioritize transactions for investigation.

This project addresses the problem by developing an end-to-end machine learning system that:

- Predicts the probability that a transaction is fraudulent.
- Identifies unusual transaction patterns using anomaly detection.
- Combines these signals into a transaction-level risk score.
- Categorizes transactions into Low, Medium, High, and Critical risk levels.
- Flags High- and Critical-risk transactions for investigation.
- Provides interactive Tableau dashboards for fraud analysis and risk intelligence.

The objective is not only to classify transactions as fraudulent or legitimate, but also to transform model outputs into actionable risk information for fraud investigation and monitoring.

4. Project Objectives

The primary objective of this project is to develop an end-to-end financial fraud detection and risk intelligence system that combines machine learning, anomaly detection, risk scoring, alert generation, and business intelligence visualization.

The key objectives are:

- Analyze transaction data to understand fraud patterns, transaction behavior, and data quality.
- Prepare and engineer relevant features from transaction, customer, time, and behavioral information.
- Develop and compare multiple supervised machine learning models for fraud classification.
- Evaluate model performance using appropriate fraud-detection metrics, including precision, recall, F1-score, ROC-AUC, confusion matrix, and cross-validation.
- Apply anomaly detection using Isolation Forest to identify unusual transaction behavior.
- Select a final supervised model based on validation results and fraud-detection requirements.
- Generate transaction-level fraud probabilities and risk scores by combining supervised prediction and anomaly signals.
- Categorize transactions into risk levels and identify High- and Critical-risk transactions requiring investigation.
- Implement an application-level alert mechanism for high-risk transactions.
- Develop interactive Tableau dashboards to communicate fraud patterns, risk distribution, trends, and investigation information.

5. Dataset Description

The project uses a financial transaction dataset containing 5,000 transaction records and 14 raw attributes. Each record represents an individual financial transaction and includes transaction details, customer information, behavioral characteristics, and the fraud target variable.

5.1 Target Variable

Fraudulent – indicates whether a transaction is classified as fraudulent:

- 0 – Non-Fraudulent
- 1 – Fraudulent

5.2 Raw Dataset Attributes

Category	Attributes
Transaction	Transaction_ID, Transaction_Date, Transaction_Amount
Customer	Customer_ID, Previous_Transactions, Average_Spend, Account_Age_Days
Transaction Context	Merchant_Category, Payment_Method, Device_Type, Location
Behavioral / Risk Indicators	Is_International, Suspicious_Keyword
Target	Fraudulent

5.3 Class Distribution

- Non-Fraudulent Transactions: 4,518 (90.36%)
- Fraudulent Transactions: 482 (9.64%)

The target distribution shows a class imbalance, with fraudulent transactions representing a smaller proportion of the dataset. This imbalance was considered during model evaluation and validation.

6. Data Quality & Exploratory Data Analysis (EDA)

6.1 Data Quality Assessment

The raw dataset was examined to understand its structure, data types, missing values, duplicate records, and consistency before beginning feature engineering and model development. The dataset contains:

- 5,000 rows
- 14 raw columns
- A binary target variable: Fraudulent
- Transaction, customer, behavioral, and contextual attributes

The data-quality assessment was performed before modeling to ensure that the dataset was suitable for further analysis.

6.2 Exploratory Data Analysis

Exploratory analysis was performed to understand transaction behavior and identify patterns associated with fraudulent transactions.

The analysis focused on:

- Fraudulent vs. non-fraudulent transaction distribution
- Transaction amount patterns
- Merchant categories
- Payment methods
- Device types
- Locations
- International transactions
- Previous transaction behavior
- Transaction timing
- Relationships between transaction amount and average customer spending

```
[13]: df["Transaction_Date"] = pd.to_datetime(
      df["Transaction_Date"],
      format="%d-%m-%y %H-%M")

      print(df["Transaction_Date"].dtype)
      print(df["Transaction_Date"].min())
      print(df["Transaction_Date"].max())

... datetime64[us]
2023-01-01 02:16:00
2024-02-21 15:34:00

[14]: print("Fraudulent value Counts:")
      print(df["Fraudulent"].value_counts())

      print("\n Fraudulent Percentage")
      print(df["Fraudulent"].value_counts(normalize=True).mul(100).round(2))

... Fraudulent value Counts:
Fraudulent
0    4518
1     482
Name: count, dtype: int64

      Fraudulent Percentage
Fraudulent
0     90.36
1      9.64
Name: proportion, dtype: float64
```

6.3 Class Distribution

The target variable contains:

Transaction Type	Count	Percentage
Non-Fraudulent	4,518	90.36%
Fraudulent	482	9.64%
Total	5,000	100%

The distribution demonstrates that fraudulent transactions form a minority class. Therefore, model evaluation emphasizes fraud-specific metrics such as precision, recall, F1-score, ROC-AUC, and confusion-matrix analysis, rather than relying on accuracy alone.

```
for col in ["Device_Type", "Location"]:
    result = (
        df.groupby(col)["Fraudulent"]
        .agg(["count", "sum", "mean"])
    )

    result["fraud_rate_%"] = (
        result["mean"] * 100
    ).round(2)

    result = result.drop(columns="mean")

    print(f"\n{'=' * 50}")
    print(f"Fraud by {col}")
    print(result.sort_values("fraud_rate_%", ascending=False))
```

[22] Python

```
=====
Fraud by Device_Type
      count  sum  fraud_rate_%
Device_Type
Desktop    1679  172      10.24
POS         1673  159       9.50
Mobile     1648  151       9.16

=====
Fraud by Location
      count  sum  fraud_rate_%
Location
Chennai    655   69      10.53
Mumbai     741   78      10.53
Bengaluru  686   71      10.35
Kolkata    713   73      10.24
Hyderabad  725   64       8.83
Pune       722   62       8.59
Delhi      758   65       8.58
```

6.4 EDA Findings

The exploratory analysis provided the foundation for subsequent feature engineering and model development. Transaction amount, customer spending behavior, transaction timing, transaction context, and other behavioral attributes were considered when constructing the machine-learning feature set.

The analysis also highlighted the importance of handling the fraud-class imbalance during model evaluation.



```
df["Transaction_Hour"] = df["Transaction_Date"].dt.hour

hourly_fraud = (
    df.groupby("Transaction_Hour")["Fraudulent"]
    .agg(["count", "sum", "mean"])
)

hourly_fraud["fraud_rate_%"] = (
    hourly_fraud["mean"] * 100
).round(2)

hourly_fraud = hourly_fraud.drop(columns="mean")

print(hourly_fraud)
```

[19]

Python

...

	Transaction_Hour	count	sum	fraud_rate_%
0	0	188	57	30.32
1	1	204	48	23.53
2	2	216	44	20.37
3	3	200	54	27.00
4	4	178	46	25.84
5	5	236	64	27.12
6	6	225	17	7.56
7	7	208	7	3.37
8	8	209	10	4.78
9	9	220	11	5.00
10	10	202	6	2.97
11	11	205	10	4.88
12	12	211	11	5.21
13	13	233	13	5.58
14	14	206	7	3.40
15	15	196	8	4.08
16	16	199	10	5.03
17	17	219	5	2.28
18	18	212	8	3.77
19	19	198	10	5.05
20	20	184	6	3.26
21	21	212	11	5.19
22	22	219	9	4.11
23	23	220	10	4.55

Feature Engineering

7.1 Feature Engineering Approach

Feature engineering was performed to transform the raw transaction attributes into additional variables that capture transaction timing, spending behavior, transaction history, and relative transaction amounts.

The engineered features were designed to provide the machine-learning models with more meaningful representations of transaction behavior.

7.2 Engineered Features

Feature	Description
Transaction_Hour	Hour extracted from the transaction timestamp
Transaction_DayOfWeek	Day of the week extracted from the transaction date
Transaction_Month	Month extracted from the transaction date
Transaction_Day	Day of the month extracted from the transaction date
Is_Weekend	Indicates whether the transaction occurred on a weekend
Amount_to_Average_Ratio	Transaction amount relative to the customer's average spending
Amount_vs_Average_Difference	Difference between transaction amount and average spending
Previous_Transactions_Band	Groups previous transaction counts into behavioral bands
Log_Amount_to_Average_Ratio	Log-transformed transaction-to-average spending ratio

7.3 Model Feature Groups

The final supervised models used the following feature groups.

Numerical Features

Transaction_Amount, Is_International, Previous_Transactions, Average_Spend, Account_Age_Days, Transaction_Hour, Transaction_DayOfWeek, Transaction_Month, Transaction_Day, Is_Weekend, Amount_to_Average_Ratio, Amount_vs_Average_Difference, Log_Amount_to_Average_Ratio

Categorical Features

Merchant_Category, Payment_Method,
Device_Type, Location, Previous_Transactions_Band

7.4 Leakage Consideration

The **Suspicious_Keyword** field was not included in the primary model feature set because it could provide information too closely related to the fraud outcome and potentially introduce target leakage.

The engineered feature **Amount_Above_Average_Flag** was also created during feature engineering but was not included in the final model feature list.

```
# Inspect the time-based features

time_features = ["Transaction_Date", "Transaction_Hour", "Transaction_DayOfWeek",
                 "Transaction_Month", "Transaction_Day", "Is_Weekend"]

print(df_fe[time_features].head(10))
```

[6] Python

...	Transaction_Date	Transaction_Hour	Transaction_DayOfWeek	\
0	2023-10-04 07:45:00	7	2	
1	2023-09-26 08:29:00	8	1	
2	2023-07-18 15:54:00	15	1	
3	2023-10-16 17:10:00	17	0	
4	2023-09-09 00:46:00	0	5	
5	2024-01-14 03:15:00	3	6	
6	2023-02-09 08:56:00	8	3	
7	2023-03-26 13:12:00	13	6	
8	2023-03-06 20:09:00	20	0	
9	2023-06-27 12:06:00	12	1	

	Transaction_Month	Transaction_Day	Is_Weekend
0	10	4	0
1	9	26	0
2	7	18	0
3	10	16	0
4	9	9	1
5	1	14	1
6	2	9	0
7	3	26	1
8	3	6	0
9	6	27	0

8. Machine Learning Methodology

8.1 Modelling Strategy

The project uses a combination of **supervised machine learning** and **unsupervised anomaly detection** to identify potentially fraudulent transactions.

The supervised learning task is treated as a **binary classification problem**, where:

- 0 = Non-Fraudulent
- 1 = Fraudulent

Multiple classification algorithms were trained and evaluated to compare their ability to identify fraudulent transactions.

8.2 Supervised Models

The following models were evaluated:

1. Logistic Regression
2. Decision Tree
3. Random Forest
4. XGBoost
5. SMOTE + Logistic Regression

The models were evaluated using fraud-focused performance metrics because the dataset contains substantially more non-fraudulent than fraudulent transactions.

```
[37] # Create Logistic Regression baseline pipeline
logistic_pipeline = Pipeline(steps=[("preprocessor", preprocessor),
                                     ("classifier", LogisticRegression(max_iter=1000, random_state=42))])
print("Logistic Regression pipeline created successfully.")
Python
... Logistic Regression pipeline created successfully.

# What does this code do?
Component.      Purpose
Pipeline        Connects preprocessing and model training
preprocessor     Scales numerical features and encodes categorical features
LogisticRegression Establishes our baseline classifier
max_iter=1000   Allows additional iterations for convergence
random_state=42 Reproducibility where applicable
markdown
Generate Code Markdown

[38] # Train the Logistic Regression model
logistic_pipeline.fit(X_train, y_train)
print("Logistic Regression model trained successfully.")
Python
... Logistic Regression model trained successfully.

# Predict class labels
y_pred = logistic_pipeline.predict(X_test)

# Predict fraud probabilities
y_proba = logistic_pipeline.predict_proba(X_test)[: , 1]

print("Predictions generated successfully.")
print("\nPredicted classes:")
print(y_pred[:10])

print("\nFraud probabilities:")
print(y_proba[:10])
Python
[39] Predictions generated successfully.
```

8.3 Model Evaluation Metrics

The following metrics were used:

- **Accuracy** — overall proportion of correctly classified transactions.
- **Precision** — proportion of transactions predicted as fraud that were actually fraudulent.
- **Recall** — proportion of actual fraudulent transactions detected by the model.
- **F1-Score** — harmonic mean of precision and recall.
- **ROC-AUC** — measures the model's ability to distinguish between fraudulent and non-fraudulent transactions.
- **Average Precision** — evaluates performance across precision-recall thresholds.
- **Confusion Matrix** — provides counts of true positives, true negatives, false positives, and false negatives.

8.4 Cross-Validation

A **5-fold Stratified Cross-Validation** approach was used to evaluate model stability across different subsets of the data while preserving the fraud/non-fraud class distribution within the folds.

The cross-validation results were used alongside the held-out test-set results when selecting the final supervised model.

8.5 Anomaly Detection

Isolation Forest was used as an additional anomaly-detection technique.

Unlike the supervised classification models, Isolation Forest identifies transactions that appear unusual based on their feature patterns. Its anomaly signal was later incorporated into the transaction-level risk scoring process.

9. Model Evaluation & Results

9.1 Test-Set Model Performance

The supervised models were evaluated on a held-out test set using fraud-focused performance metrics.

Model	Accuracy	Fraud Precision	Fraud Recall	Fraud F1	ROC-AUC
Logistic Regression	89.60%	35.71%	10.42%	16.13%	0.7454
Decision Tree	73.30%	21.97%	69.79%	33.42%	0.6784
Random Forest	74.30%	23.26%	72.92%	35.26%	0.7586
XGBoost	76.20%	22.48%	60.42%	32.77%	0.7561

9.2 Cross-Validation Results

Five-fold stratified cross-validation was used to assess model performance and stability.

Model	ROC-AUC Mean	AP Mean	Precision Mean	Recall Mean	F1 Mean
Logistic Regression	0.8028	0.3173	0.5025	0.1307	0.2069
Decision Tree	0.7770	0.2788	0.2585	0.7884	0.3892
Random Forest	0.8120	0.3191	0.2696	0.8299	0.4069
XGBoost	0.8160	0.3194	0.2736	0.6992	0.3931
SMOTE Logistic Regression	0.8015	0.3174	0.2379	0.7656	0.3628

```
# Import required libraries
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.metrics import make_scorer, precision_score, recall_score, f1_score
import pandas as pd

# Define the existing model pipelines
model_pipelines = {
    "Logistic Regression": logistic_pipeline, "Decision Tree": decision_tree_pipeline,
    "Random Forest": random_forest_pipeline, "XGBoost": xgboost_pipeline, "SMOTE Logistic Regression": smote_logistic_pipeline
}

# Create stratified cross-validation
# Stratification preserves the approximate fraud/non-fraud ratio in each fold
stratified_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Define evaluation metrics
scoring_metrics = {
    "ROC_AUC": "roc_auc", "Average_Precision": "average_precision",
    "Precision": make_scorer(precision_score, zero_division=0), "Recall": make_scorer(recall_score, zero_division=0),
    "F1_Score": make_scorer(f1_score, zero_division=0)
}

# Store cross-validation results
cross_validation_results = []

# Evaluate each model pipeline
for model_name, pipeline in model_pipelines.items():
    # Perform cross-validation on the complete pipeline
    cv_results = cross_validate(estimator=pipeline, X=X, y=y, cv=stratified_cv, scoring=scoring_metrics, n_jobs=-1)

    # Store the average and standard deviation of each metric
    cross_validation_results.append({
        "Model": model_name, "ROC_AUC_Mean": cv_results["test_ROC_AUC"].mean(), "ROC_AUC_Std": cv_results["test_ROC_AUC"].std(),
        "Average_Precision_Mean": cv_results["test_Average_Precision"].mean(), "Average_Precision_Std": cv_results["test_Average_Precision"].std(),
        "Precision_Mean": cv_results["test_Precision"].mean(), "Recall_Mean": cv_results["test_Recall"].mean(), "F1_Score_Mean": cv_results["test_F1_Score"].mean()
    })

# Convert the results into a DataFrame
cross_validation_df = pd.DataFrame(cross_validation_results)

# Display the results
cross_validation_df.round(4)
```

	Model	ROC_AUC_Mean	ROC_AUC_Std	Average_Precision_Mean	Average_Precision_Std	Precision_Mean	Recall_Mean	F1_Score_Mean
0	Logistic Regression	0.8028	0.0076	0.3173	0.0123	0.5025	0.1307	0.2069
1	Decision Tree	0.7770	0.0183	0.2788	0.0108	0.2585	0.7884	0.3892
2	Random Forest	0.8120	0.0035	0.3191	0.0300	0.2696	0.8299	0.4069
3	XGBoost	0.8160	0.0096	0.3194	0.0143	0.2736	0.6992	0.3931
4	SMOTE Logistic Regression	0.8015	0.0100	0.3174	0.0252	0.2379	0.7656	0.3628

9.3 Model Selection

Random Forest was selected as the final supervised model based on the overall validation results, particularly its cross-validation recall and F1-score, together with its relatively low ROC-AUC variation across folds.

XGBoost achieved a slightly higher mean ROC-AUC and Average Precision, but Random Forest achieved higher cross-validation recall and F1-score.

Therefore, the final system uses Random Forest for transaction-level fraud probability generation.

```
# Import pandas for creating the model comparison table
import pandas as pd

# Import evaluation metrics
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix)

# Store the prediction results from each trained model
models = { "Logistic Regression": (y_pred, y_proba), "Decision Tree": (dt_y_pred, dt_y_proba),
           "Random Forest": (rf_y_pred, rf_y_proba), "XGBoost": (xgb_y_pred, xgb_y_proba) }

# Create an empty list to store calculated model results
comparison_results = []

# Calculate the evaluation metrics for every model
for model_name, (predictions, probabilities) in models.items():

    # Calculate the confusion matrix
    tn, fp, fn, tp = confusion_matrix(y_test, predictions).ravel()

    # Calculate performance metrics from the actual predictions
    accuracy = accuracy_score(y_test, predictions)
    precision = precision_score(y_test, predictions, zero_division=0)
    recall = recall_score(y_test, predictions, zero_division=0)
    f1 = f1_score(y_test, predictions, zero_division=0)
    roc_auc = roc_auc_score(y_test, probabilities)

    # Store the calculated results
    comparison_results.append([
        "Model": model_name, "Accuracy": accuracy * 100, "Fraud Precision": precision * 100, "Fraud Recall": recall * 100,
        "Fraud F1": f1 * 100, "ROC-AUC": roc_auc * 100, "False Positives": fp, "False Negatives": fn])

# Convert the results into a DataFrame
model_comparison_df = pd.DataFrame(comparison_results)

# Format percentage-based metrics
percentage_columns = ["Accuracy", "Fraud Precision", "Fraud Recall", "Fraud F1", "ROC-AUC"]

for column in percentage_columns:
    model_comparison_df[column] = model_comparison_df[column].map(lambda x: f"{x:.2f}%")

# Display the final comparison table
display(model_comparison_df)
```

	Model	Accuracy	Fraud Precision	Fraud Recall	Fraud F1	ROC-AUC	False Positives	False Negatives
0	Logistic Regression	89.60%	35.71%	10.42%	16.13%	74.54%	18	86
1	Decision Tree	73.30%	21.97%	69.79%	33.42%	67.84%	238	29
2	Random Forest	74.30%	23.28%	72.92%	35.26%	75.86%	231	26
3	XGBoost	76.20%	22.48%	60.42%	32.77%	75.61%	200	38

9.4 Confusion Matrix & Error Analysis

On the held-out test set, the Random Forest model produced:

- **False Negatives: 26**
- **False Positives: 231**

False negatives represent fraudulent transactions that were not detected, while false positives represent legitimate transactions incorrectly flagged as fraudulent.

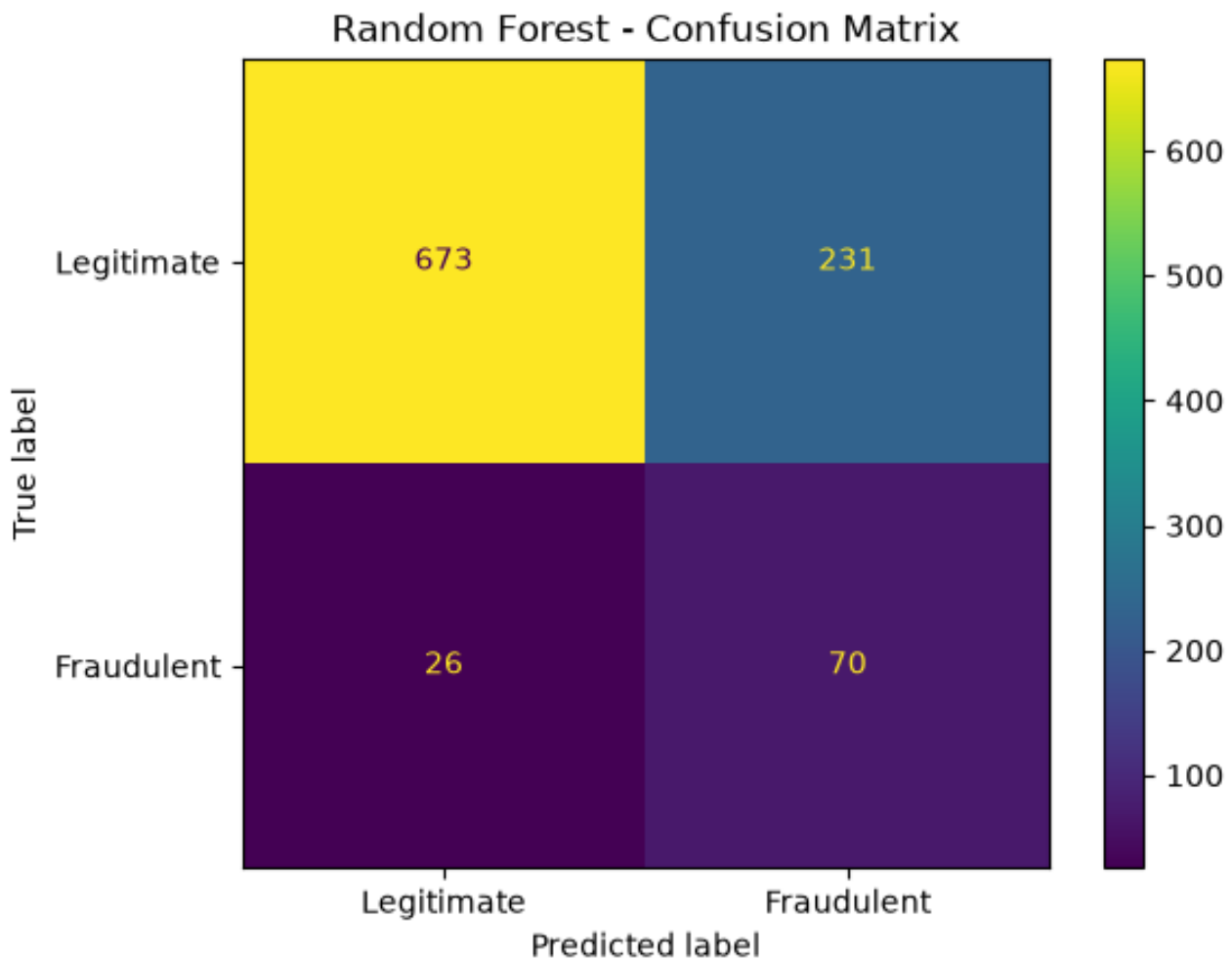
These errors were considered important because fraud detection requires balancing the cost of missed fraud against the operational workload created by false alerts.

```
## Generate the Confusion matrix
from sklearn.metrics import confusion_matrix

rf_cm = confusion_matrix(y_test, rf_y_pred)

print("Random Forest Confusion Matrix:")
print(rf_cm)
```

```
Random Forest Confusion Matrix:
[[673 231]
 [ 26  70]]
```



10. Threshold & Error Analysis

10.1 Why Threshold Analysis Was Required

The default classification threshold is not always optimal for fraud detection because the cost of missing a fraudulent transaction can differ from the cost of investigating a legitimate transaction.

Threshold analysis was therefore performed to understand how changing the decision threshold affects:

- Precision
- Recall
- F1-score
- False positives
- False negatives

10.2 Fraud Detection Trade-Off

A lower classification threshold generally allows more transactions to be flagged as potentially fraudulent, which can increase fraud detection but may also increase false-positive alerts.

A higher threshold can reduce false-positive alerts but may allow more fraudulent transactions to remain undetected.

Therefore, threshold selection must consider the operational requirements of the fraud investigation process.

10.3 False Positive and False Negative Analysis

Two types of classification errors are particularly important:

- **False Positive (FP)**

A legitimate transaction is incorrectly identified as fraudulent.

- **False Negative (FN)**

A fraudulent transaction is incorrectly classified as non-fraudulent.

For this project, both error types were analyzed because they have different operational implications.

- False negatives represent potentially missed fraudulent transactions.
- False positives increase the number of transactions requiring investigation.

The final system therefore uses the model's **fraud probability** as an input to the downstream risk-scoring framework rather than treating a single classification threshold as the only decision mechanism.

```
# Create a detailed error analysis dataset for Random Forest
# Copy the original test transactions using their original indices
rf_error_analysis_df = df.loc[X_test.index].copy()

# Add actual target values
rf_error_analysis_df["Actual_Fraudulent"] = y_test

# Add Random Forest predictions
rf_error_analysis_df["Predicted_Fraudulent"] = rf_y_pred

# Add Random Forest fraud probabilities
rf_error_analysis_df["Fraud_Probability"] = rf_y_prob

# Classify each prediction outcome
rf_error_analysis_df["Prediction_Type"] = "Correct"

# Identify false positives
rf_error_analysis_df.loc[(rf_error_analysis_df["Actual_Fraudulent"] == 0) & (rf_error_analysis_df["Predicted_Fraudulent"] == 1), "Prediction_Type"] = "False Positive"

# Identify false negatives
rf_error_analysis_df.loc[(rf_error_analysis_df["Actual_Fraudulent"] == 1) & (rf_error_analysis_df["Predicted_Fraudulent"] == 0), "Prediction_Type"] = "False Negative"

# Identify true positives
rf_error_analysis_df.loc[(rf_error_analysis_df["Actual_Fraudulent"] == 1) & (rf_error_analysis_df["Predicted_Fraudulent"] == 1), "Prediction_Type"] = "True Positive"

# Identify true negatives
rf_error_analysis_df.loc[(rf_error_analysis_df["Actual_Fraudulent"] == 0) & (rf_error_analysis_df["Predicted_Fraudulent"] == 0), "Prediction_Type"] = "True Negative"

# Display the count of each prediction type
print("Random Forest Prediction Type Counts:")
print(rf_error_analysis_df["Prediction_Type"].value_counts())

# Display the percentage of each prediction type
print("\nRandom Forest Prediction Type Percentages:")
print(rf_error_analysis_df["Prediction_Type"].value_counts(normalize=True).mul(100).round(2))
```

✓ 0.0s Python

```
...
Random Forest Prediction Type Counts:
Prediction_Type
True Negative    673
False Positive   231
True Positive     70
False Negative    26
Name: count, dtype: int64

Random Forest Prediction Type Percentages:
Prediction_Type
True Negative    67.3
False Positive   23.1
True Positive     7.0
False Negative    2.6
Name: proportion, dtype: float64
```

11. Anomaly Detection

11.1 Isolation Forest

In addition to supervised fraud classification, the project uses **Isolation Forest** to identify transactions that exhibit unusual patterns within the feature space.

Isolation Forest provides an independent anomaly signal that complements the supervised fraud probability.

11.2 Anomaly Detection Results

```
# Calculate anomaly detection metrics

# Import evaluation metrics from scikit-learn
from sklearn.metrics import precision_score, recall_score, f1_score

# Convert anomaly predictions into binary fraud-style predictions
# -1 (Anomaly) becomes 1, and 1 (Normal) becomes 0
anomaly_binary_predictions = (anomaly_predictions == -1).astype(int)

# Calculate precision for anomaly detection
anomaly_precision = precision_score(y, anomaly_binary_predictions, zero_division=0)

# Calculate recall for anomaly detection
anomaly_recall = recall_score(y,
                             anomaly_binary_predictions,
                             zero_division=0
                            )

# Calculate F1-score for anomaly detection
anomaly_f1 = f1_score(y,
                     anomaly_binary_predictions,
                     zero_division=0
                    )

# Display the calculated metrics
print("Isolation Forest Anomaly Detection Metrics")
print("-----")
print(f"Precision: {anomaly_precision:.4f}")
print(f"Recall:    {anomaly_recall:.4f}")
print(f"F1-Score:  {anomaly_f1:.4f}")
```

[87] ✓ 0.0s Python

```
... Isolation Forest Anomaly Detection Metrics
-----
Precision: 0.2020
Recall:    0.2095
F1-Score:  0.2057
```

The Isolation Forest model identified:

- **500 transactions as anomalies**
- **4,500 transactions as normal**
- **Anomaly proportion: 10%**

The anomaly-detection evaluation produced:

Metric	Result
Precision	20.20%
Recall	20.95%
F1-Score	20.57%

11.3 Interpretation

The anomaly signal does **not** indicate that a transaction is confirmed fraudulent.

Instead:

- **Anomaly_Flag = 1** indicates that Isolation Forest classified the transaction as anomalous.
- **Anomaly_Flag = 0** indicates that the transaction was not classified as anomalous.

This signal is used together with the supervised Random Forest fraud probability in the downstream risk-scoring framework.

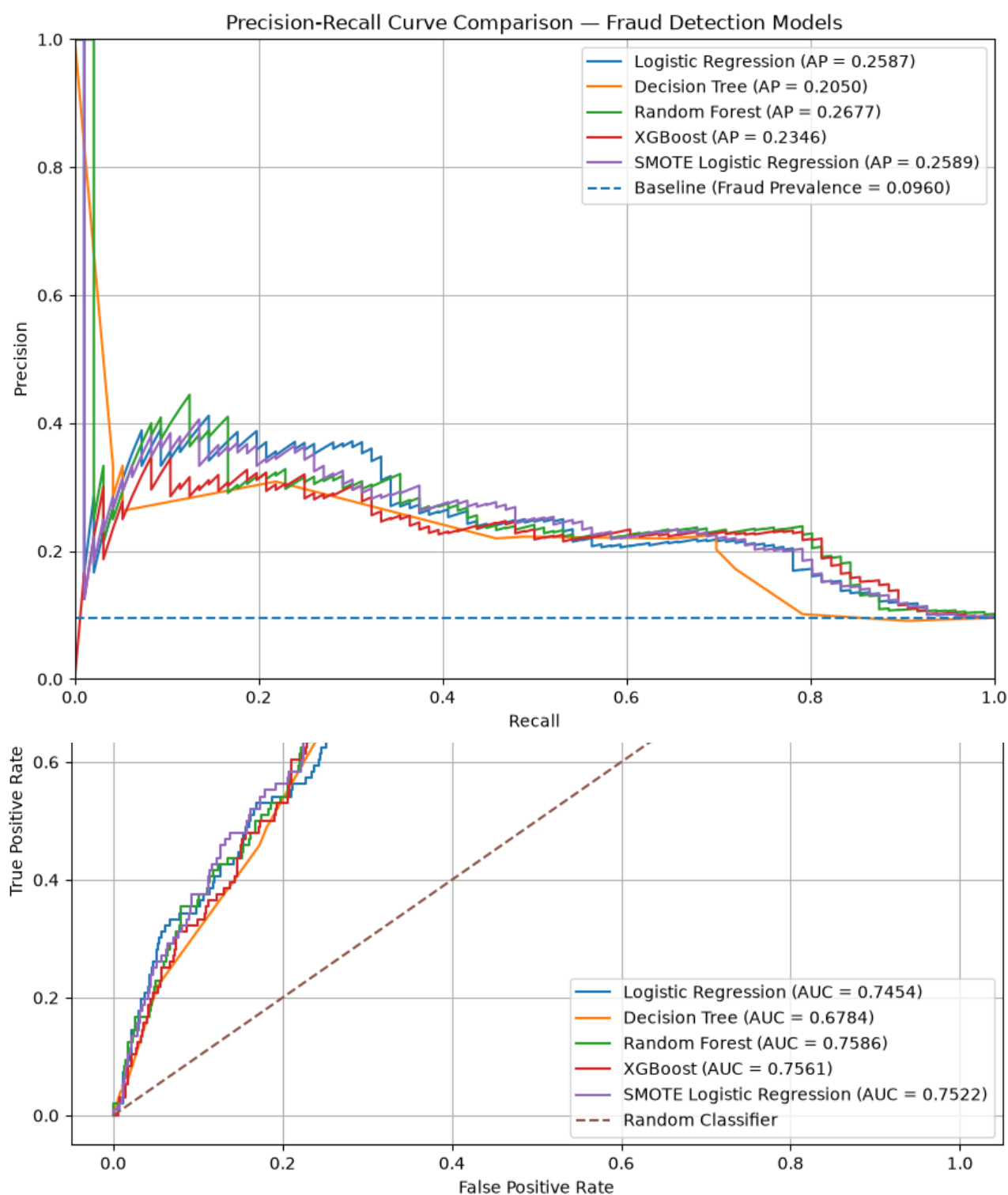
12. Final Model Selection

12.1 Selected Supervised Model

Based on the test-set evaluation and 5-fold stratified cross-validation results, **Random Forest** was selected as the final supervised fraud-classification model.

The selection considered multiple performance dimensions rather than relying on a single metric.

12.2 Selection Rationale



Random Forest achieved:

- **Cross-validation ROC-AUC: 0.8120**
- **Cross-validation Recall: 0.8299**
- **Cross-validation F1-Score: 0.4069**
- **ROC-AUC standard deviation: 0.0035**

The model also achieved a **72.92% fraud recall** and **34.99% fraud F1-score** on the held-out test set.

XGBoost achieved a slightly higher cross-validation ROC-AUC of **0.8160** and Average Precision of **0.3194**, but Random Forest achieved higher cross-validation recall and F1-score.

Therefore, Random Forest was selected for the final risk-scoring pipeline because the project places particular importance on detecting fraudulent transactions while maintaining a practical balance between detection and investigation workload.

12.3 Final Model Role

The selected Random Forest model is retrained using the complete available feature dataset and generates a fraud probability for each transaction.

This probability becomes the primary supervised signal used by the downstream Risk Scoring component.

13. Risk Scoring

13.1 Risk Scoring Framework

The final risk score combines two signals:

1. **Random Forest fraud probability**
2. **Isolation Forest anomaly signal**

The Random Forest probability provides the primary fraud-risk signal, while the anomaly flag adds an independent indicator of unusual transaction behavior.

13.2 Risk Score Calculation

The final risk score is calculated using:

Risk Score = (80% × Fraud Probability Score) + (20% × Anomaly Score)

Where:

- **Fraud Probability Score** = Random Forest fraud probability × 100
- **Anomaly Score** = 100 when Anomaly_Flag = 1, otherwise 0

The resulting score is constrained to a **0–100 range**.

The **80/20 weighting** is an operational design choice used to combine the two signals; it is not learned by the machine-learning model.

```
# Create the final risk score by combining supervised and anomaly signals
# Convert the anomaly flag into a 0-100 anomaly score
anomaly_score = anomaly_binary_predictions * 100

# Combine the two signals, Random Forest probability contributes 80%, Isolation Forest anomaly signal contributes 20%
final_risk_score = (0.80 * base_risk_score + 0.20 * anomaly_score)

# Keep the score within the 0-100 range
final_risk_score = np.clip(final_risk_score, 0, 100)

# Display basic statistics
print("Final Risk Score Statistics")
print("-----")
print(f"Minimum: {final_risk_score.min():.2f}")
print(f"Maximum: {final_risk_score.max():.2f}")
print(f"Mean: {final_risk_score.mean():.2f}")

[104] ✓ 0.0s Python
```

```
Final Risk Score Statistics
-----
Minimum: 11.11
Maximum: 89.93
Mean: 30.63

# Assign an operational risk level based on the final risk score
def assign_risk_level(score):
    # Low risk # Medium risk # High risk # Critical risk
    if score < 25: return "Low"
    elif score < 50: return "Medium"
    elif score < 75: return "High"
    else: return "Critical"

# Apply the risk-level rules to all transactions
risk_level = np.array([assign_risk_level(score)
                       for score in final_risk_score])

# Display the risk-level distribution
print("Risk Level Distribution")
print("-----")
print(pd.Series(risk_level).value_counts().sort_index())

[105] ✓ 0.0s Python
```

```
Risk Level Distribution
-----
Critical    105
High       753
Low       3154
Medium     988
Name: count, dtype: int64
```

13.3 Risk Levels

Risk Score	Risk Level
0-24.99	Low
25-49.99	Medium
50-74.99	High
75-100	Critical

13.4 Risk Distribution

Across the 5,000 transactions:

Risk Level	Transactions
Low	3,154
Medium	988
High	753
Critical	105

Total	5,000
-------	-------

Therefore, **858 transactions** were classified as High or Critical risk and marked for further investigation.

13.5 Interpretation

The Risk Score provides a prioritisation mechanism for investigation.

A High or Critical risk score indicates **elevated risk**, but it does not by itself confirm that a transaction is fraudulent. Final investigation and verification remain necessary.

14. Alert Mechanism

14.1 Alert Workflow

The alert mechanism converts elevated transaction risk into an actionable investigation queue.

The workflow operates as follows:

1. The Random Forest model generates a fraud probability.
2. Isolation Forest provides an anomaly signal.
3. The two signals are combined into the final Risk Score.
4. Each transaction is assigned a Risk Level: **Low, Medium, High, or Critical**.
5. Transactions classified as **High or Critical** are marked as requiring investigation.
6. An alert record is created containing the transaction and risk information.
7. Each alert receives an **Alert ID** and an **Alert Severity**.
8. The alert status initially remains **PENDING_REVIEW** until investigation takes place.

14.2 Alert Volume

The completed risk-scoring system identified:

- **High-risk transactions: 753**
- **Critical-risk transactions: 105**
- **Total High + Critical alerts: 858**

```
# Create a summary of the alert queue
alert_summary = (
    alert_log
    .groupby(["Risk_Level", "Alert_Severity", "Alert_Status"])
    .size()
    .reset_index(name="Alert_Count")
)

# Display the alert summary
print("Alert Summary")
print("-----")
display(alert_summary)
```

[121] ✓ 0.0s Python

... Alert Summary

	Risk_Level	Alert_Severity	Alert_Status	Alert_Count
0	Critical	CRITICAL	PENDING_REVIEW	105
1	High	HIGH	PENDING_REVIEW	753

14.3 Alert Information

Each alert contains relevant investigation information, including:

- Alert ID
- Transaction ID
- Customer ID
- Transaction Amount
- Location
- Device Type
- Fraud Probability
- Anomaly Flag
- Risk Score
- Risk Level
- Alert Severity
- Alert Status

14.4 Current Implementation

```
# Display the highest-priority alert from the alert queue
top_alert = alert_log.iloc[0]

print("=" * 60)
print("🚨 FRAUD RISK ALERT")
print("=" * 60)

# Ensure Alert_ID exists, even if the alert-queue cell was not rerun
if "Alert_ID" not in alert_log.columns:
    alert_log = alert_log.reset_index(drop=True).copy()
    alert_log["Alert_ID"] = [
        f"ALERT_{i:04d}" for i in range(1, len(alert_log) + 1)]

# Refresh the selected alert after adding Alert_ID
top_alert = alert_log.iloc[0]
print(f"Alert ID           : {top_alert['Alert_ID']}")
print(f"Transaction ID      : {top_alert['Transaction_ID']}")
print(f"Customer ID         : {top_alert['Customer_ID']}")
print(f"Transaction Amount   : {top_alert['Transaction_Amount']:.2f}")
print(f"Location            : {top_alert['Location']}")
print(f"Device Type         : {top_alert['Device_Type']}")
print(f"Fraud Probability    : {top_alert['Fraud_Probability']:.2%}")
print(f"Anomaly Flag        : {top_alert['Anomaly_Flag']}")
print(f"Risk Score          : {top_alert['Risk_Score']:.2f}/100")
print(f"Risk Level          : {top_alert['Risk_Level']}")
print(f"Alert Severity      : {top_alert['Alert_Severity']}")
print(f"Alert Status        : {top_alert['Alert_Status']}")

print("=" * 80)
print("ACTION: Review this transaction according to the fraud investigation procedure.")
print("=" * 80)
```

[123] ✓ 0.0s Python

```
=====
🚨 FRAUD RISK ALERT
=====
Alert ID           : ALERT_0001
Transaction ID      : T100019
Customer ID        : CUST2634
Transaction Amount  : 27.54
Location           : Hyderabad
Device Type        : POS
Fraud Probability   : 63.35%
Anomaly Flag       : 0
Risk Score         : 50.68/100
Risk Level         : High
Alert Severity     : HIGH
Alert Status       : PENDING_REVIEW
=====
ACTION: Review this transaction according to the fraud investigation procedure.
=====
```

The current implementation provides an application-level alert log and investigation queue.

The system does not automatically send email or SMS notifications. The alert output can be integrated with an organization's notification or fraud-management system in a production environment.

An alert represents elevated fraud risk and does not independently confirm that a transaction is fraudulent.

15. Tableau Analytics

15.1 Tableau Dashboard Development

The processed transaction dataset was prepared for business intelligence analysis and connected to Tableau Public.

The Tableau environment provides interactive views of transaction fraud, risk levels, transaction patterns, and investigation information.

The dashboard solution consists of three main dashboards.

15.2 Dashboard 1 — Fraud Risk Executive Overview

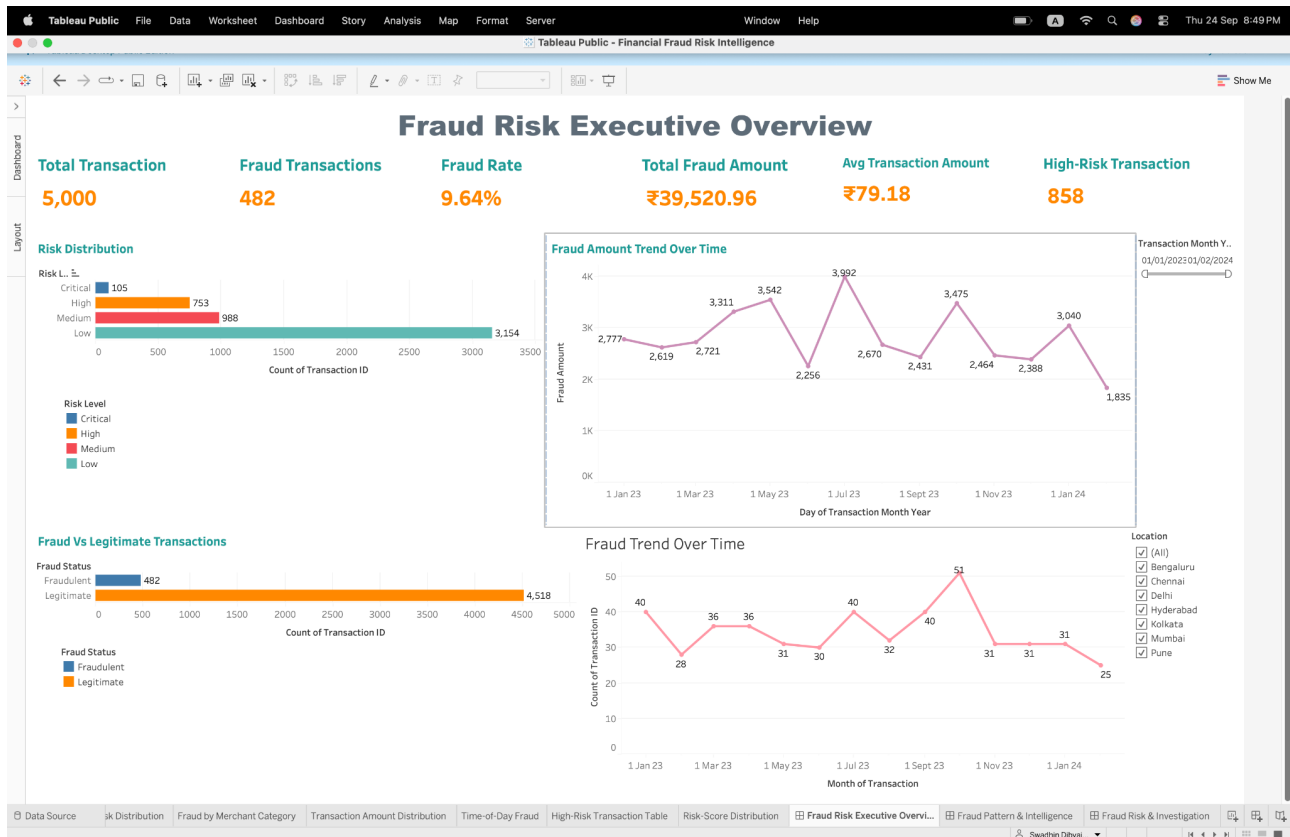
This dashboard provides a high-level view of the fraud-risk landscape.

Key components include:

- Total Transactions
- Fraud Transactions
- Fraud Rate
- Total Fraud Amount
- Average Transaction Amount

- High-Risk Transactions
- Risk Distribution
- Fraud Trend Over Time
- Fraud Amount Trend Over Time
- Fraud vs. Legitimate Transactions

Interactive filters include transaction period and location.



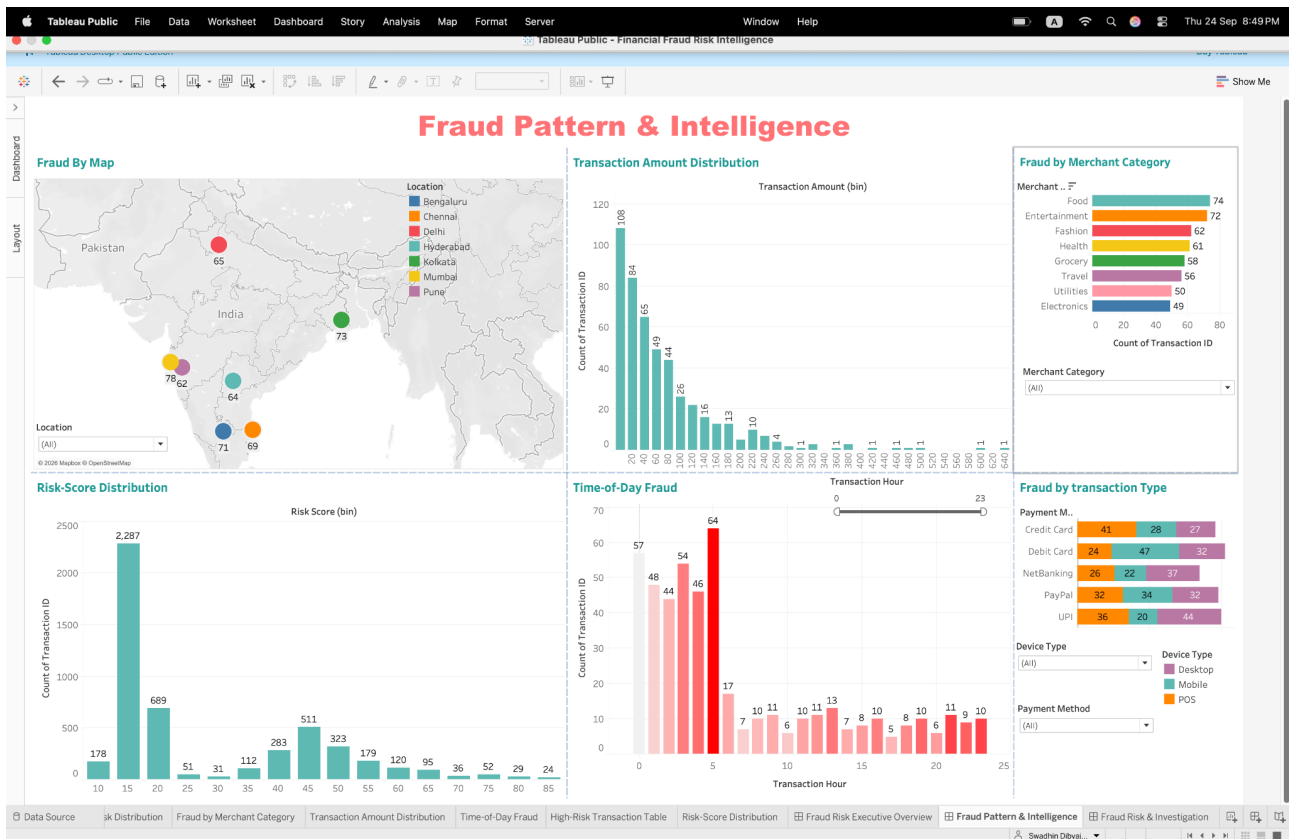
15.3 Dashboard 2 – Fraud Pattern & Intelligence

This dashboard focuses on identifying transaction and fraud patterns.

Key visualizations include:

- Fraud by Location
- Transaction Amount Distribution
- Fraud by Merchant Category
- Risk-Score Distribution
- Time-of-Day Fraud
- Fraud by Transaction Type

Interactive filters allow users to explore fraud patterns across relevant transaction attributes.



15.4 Dashboard 3 – Fraud Risk & Investigation

This dashboard focuses on high-risk transaction investigation.

Key components include:

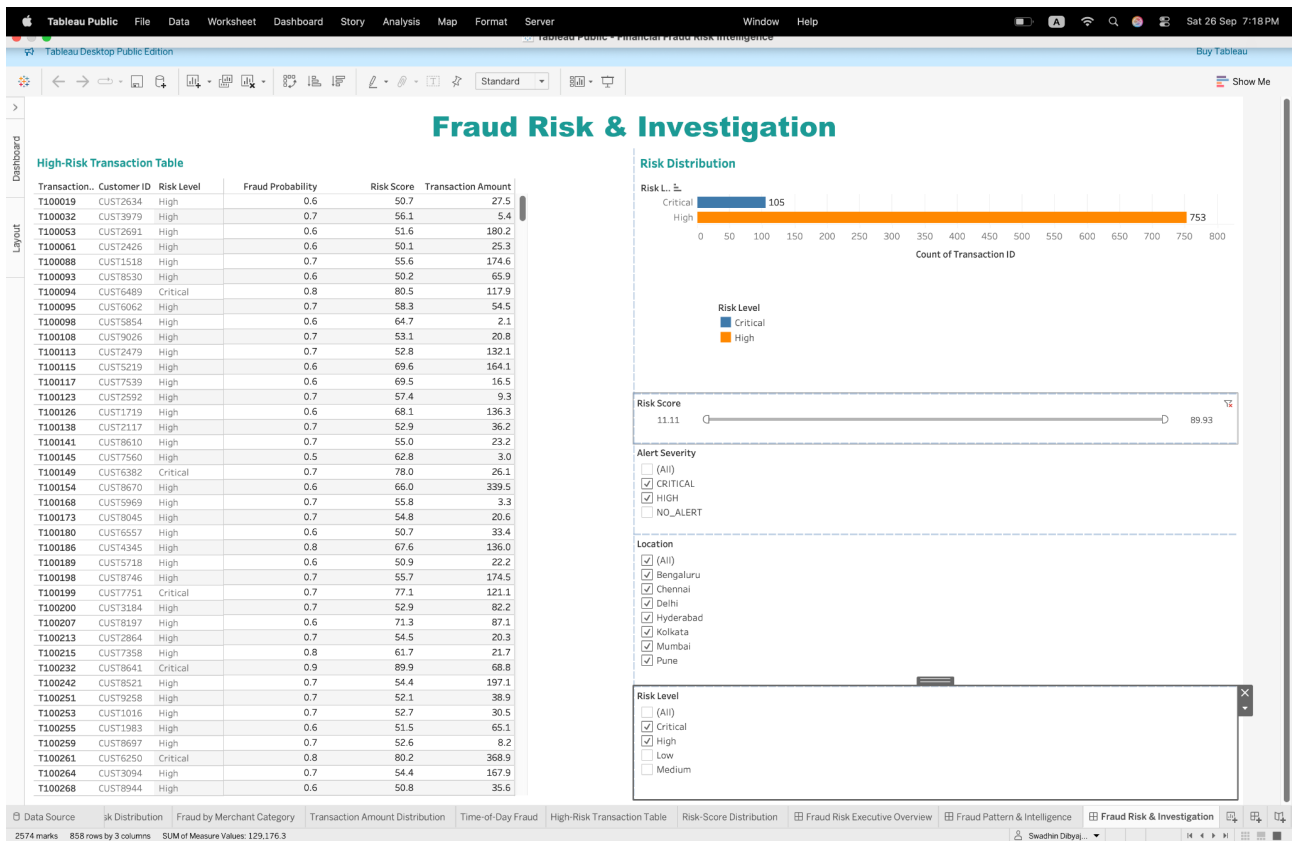
- High-Risk Transaction Table
- Risk Distribution
- Risk Level filtering
- Alert Severity filtering
- Location filtering
- Risk Score filtering

The high-risk transaction table provides transaction-level information such as transaction amount, fraud probability, and risk score to support investigation prioritization.

15.5 Tableau Data Preparation

A dedicated processed dataset named: `tableau_dashboard_data.csv` was created from the feature-engineered transaction data.

The dataset contains the transaction information, fraud status, fraud probability, anomaly flag, risk score, risk level, and alert-related fields required for dashboard development.



16. Business Insights

16.1 Fraud and Risk Overview

The completed system analyzed **5,000 transactions**, of which **482 (9.64%)** were labeled fraudulent.

The risk-scoring framework classified the transactions into four risk levels:

- **Low: 3,154**
- **Medium: 988**
- **High: 753**
- **Critical: 105**

A total of **858 transactions** were therefore classified as High or Critical risk and marked for investigation.

16.2 Fraud Detection Performance

The Random Forest model was selected as the final supervised model after comparison with the other evaluated models.

Its held-out test-set results included:

- **Fraud Recall: 72.92%**
- **Fraud F1-Score: 34.99%**
- **ROC-AUC: 0.7586**

Five-fold cross-validation produced a mean recall of **82.99%** and mean F1-score of **40.69%**.

These results demonstrate the model's ability to identify a substantial proportion of fraudulent transactions while also highlighting the presence of false-positive alerts.

16.3 Combined Risk Intelligence

The project extends beyond binary fraud classification by combining:

Fraud Probability + Anomaly Signal → Risk Score → Risk Level → Alert

This provides a structured method for prioritizing transactions for investigation rather than treating every transaction prediction as an independent yes/no decision.

16.4 Dashboard-Based Investigation

The Tableau dashboards allow users to examine:

- Fraud trends over time
- Fraud amount patterns
- Fraud by merchant category
- Fraud by payment method and device type
- Geographic fraud patterns
- Transaction amount distributions
- Time-of-day fraud patterns
- Risk-level distributions
- High-risk transactions

These views provide multiple perspectives for analyzing transaction risk and supporting fraud investigation.

17. Limitations

17.1 Dataset Limitations

The project was developed using a dataset containing **5,000 transaction records**. Therefore, the observed patterns and model results are specific to this dataset and may not represent the behavior of a larger real-world financial transaction population.

17.2 Class Imbalance

Fraudulent transactions represent **9.64%** of the dataset. This class imbalance makes accuracy alone insufficient for evaluating fraud detection performance and increases the importance of fraud-specific metrics such as recall, precision, and F1-score.

17.3 Model Limitations

The supervised models are trained using the available transaction attributes and engineered features. Their performance may change when applied to transaction data with different behavioral patterns, distributions, or fraud characteristics.

The Random Forest model also produces false positives and false negatives. Therefore, model predictions should support investigation rather than replace human verification.

17.4 Anomaly Detection Limitations

Isolation Forest identifies unusual transaction patterns, but an anomaly is **not necessarily fraudulent**.

The anomaly signal should therefore be interpreted as an additional risk indicator rather than direct evidence of fraud.

17.5 Risk-Scoring Design

The **80/20 weighting** between fraud probability and anomaly signal, as well as the Low/Medium/High/Critical risk bands, are operational design choices created for this project.

They are not automatically learned or optimized by the machine-learning models.

17.6 Alert Mechanism

The current system provides an **application-level alert log and investigation queue**. Automated email or SMS notification was not implemented in the final system.

A production implementation would require integration with an organization's notification and fraud-management infrastructure.

17.7 Deployment Scope

The current project focuses on the completed machine-learning, risk-intelligence, alert, and Tableau analytics workflow.

A production-grade real-time deployment would require additional infrastructure for model serving, authentication, monitoring, data pipelines, security, and system integration.

18. Future Improvements

18.1 Real-Time Prediction API

The model could be deployed through an API service such as FastAPI to support real-time transaction scoring.

A future API workflow could accept transaction information and return:

- Fraud probability
- Anomaly signal
- Risk score
- Risk level
- Alert status

18.2 Automated Notification Integration

The current application-level alert mechanism could be connected to production notification systems such as:

- Email
- SMS
- Internal fraud-management platforms
- Security monitoring systems

18.3 Model Monitoring

A production system should continuously monitor:

- Model performance
- Prediction distributions
- Data quality
- Feature drift
- Fraud-rate changes
- False-positive and false-negative rates

18.4 Model Retraining

As new transaction and investigation outcomes become available, the model could be periodically retrained to adapt to changing transaction behavior and emerging fraud patterns.

18.5 Advanced Risk Scoring

Future versions could investigate data-driven optimization of the risk-score weights and risk thresholds using historical investigation outcomes rather than relying on manually defined operational weights.

18.6 Expanded Feature Engineering

Additional behavioral features could be introduced when suitable data becomes available, such as:

- Customer transaction frequency
- Recent spending changes
- Merchant-level behavioral patterns
- Device history
- Location changes
- Customer transaction sequences

18.7 Production Deployment

The completed analytical workflow could be extended into a production architecture containing:

Data Pipeline → Model API → Risk Engine → Alert System → Dashboard → Monitoring

This would allow the project to move from an analytical prototype toward a continuously operating fraud-risk intelligence platform.

19. Conclusion

19.1 Project Conclusion

The **Financial Fraud Detection & Risk Intelligence System** was developed as an end-to-end machine-learning and analytics workflow for identifying potentially fraudulent transactions and prioritizing transaction risk.

The project combined:

Data Quality & EDA → Feature Engineering → Supervised Machine Learning → Anomaly Detection → Model Validation → Risk Scoring → Alert Mechanism → Tableau Analytics

Multiple supervised classification models were evaluated, with **Random Forest** selected as the **final model** based on the overall validation results and its fraud-detection performance.

The addition of **Isolation Forest** provided an independent anomaly signal, allowing the system to consider both predicted fraud probability and unusual transaction behavior.

These signals were combined into a transaction-level **Risk Score**, which categorized transactions into Low, Medium, High, and Critical risk levels. High- and Critical-risk transactions were then identified through the application-level alert mechanism for further investigation.

Finally, the processed results were presented through three interactive **Tableau dashboards**, providing views of fraud trends, transaction patterns, risk distribution, and high-risk transactions.

Overall, the project demonstrates the complete transformation of raw transaction data into **machine-learning predictions, risk intelligence, investigation alerts, and business-facing analytics**.